

Security & Authentication

▼ Relevant source files

Backend/STUDER/src/main/java/facu/studer/security/JwtAuthenticationFilter.java

Backend/STUDER/src/main/java/facu/studer/security/JwtUtil.java

Backend/STUDER/src/main/java/facu/studer/security/LoginRateLimiter.java

Backend/STUDER/src/main/java/facu/studer/security/SecurityConfig.java

Backend/STUDER/src/main/java/facu/studer/security/SecurityUtils.java

Backend/STUDER/src/main/resources/application-dev.properties

Frontend/STUDER/src/app/core/auth/auth-api.service.ts

Frontend/STUDER/src/app/core/auth/auth-state.service.ts

Frontend/STUDER/src/app/features/home/home.component.ts

The STUDER security architecture implements a stateless authentication system based on **JSON Web Tokens (JWT)** and **HttpOnly Refresh Cookies**. This ensures secure communication between the Angular frontend and the Spring Boot backend while maintaining scalability and protecting against common vulnerabilities like CSRF and XSS.

Core Configuration

The security layer is centered around the `SecurityConfig` class, which defines the HTTP filter chain, CORS policies, and endpoint accessibility.

SecurityConfig

The system uses `stateless` session management

Backend/STUDER/src/main/java/facu/studer/security/SecurityConfig.java 74-75 meaning no

session state is stored on the server. Security is enforced by the `JwtAuthenticationFilter`,

which is injected into the filter chain before the standard `UsernamePasswordAuthenticationFilter`

Backend/STUDER/src/main/java/facu/studer/security/SecurityConfig.java 89

- **Public Routes:** `/api/v1/auth/**`, `/api/v1/users/register`, and Swagger documentation

Backend/STUDER/src/main/java/facu/studer/security/SecurityConfig.java 80-83

- **Protected Routes:** All other requests require a valid JWT

Backend/STUDER/src/main/java/facu/studer/security/SecurityConfig.java 85

- **Password Encoding:** Uses `BCryptPasswordEncoder` for hashing user credentials

Backend/STUDER/src/main/java/facu/studer/security/SecurityConfig.java 55-57

CORS Policy

Cross-Origin Resource Sharing is configured to allow specific origins (defaulting to `localhost:3000` and `localhost:4200`) and exposes the `Authorization` header so the frontend can read the access tokens

Backend/STUDER/src/main/java/facu/studer/security/SecurityConfig.java 131-165

Sources: Backend/STUDER/src/main/java/facu/studer/security/SecurityConfig.java 1-166

Backend/STUDER/src/main/resources/application-dev.properties 9-11

Authentication Flow

STUDER utilizes a dual-token strategy:

1. **Access Token:** Short-lived (default 15 mins), sent in the `Authorization: Bearer <token>`

header Backend/STUDER/src/main/java/facu/studer/security/JwtUtil.java 23-25

Backend/STUDER/src/main/java/facu/studer/security/JwtAuthenticationFilter.java 37-40

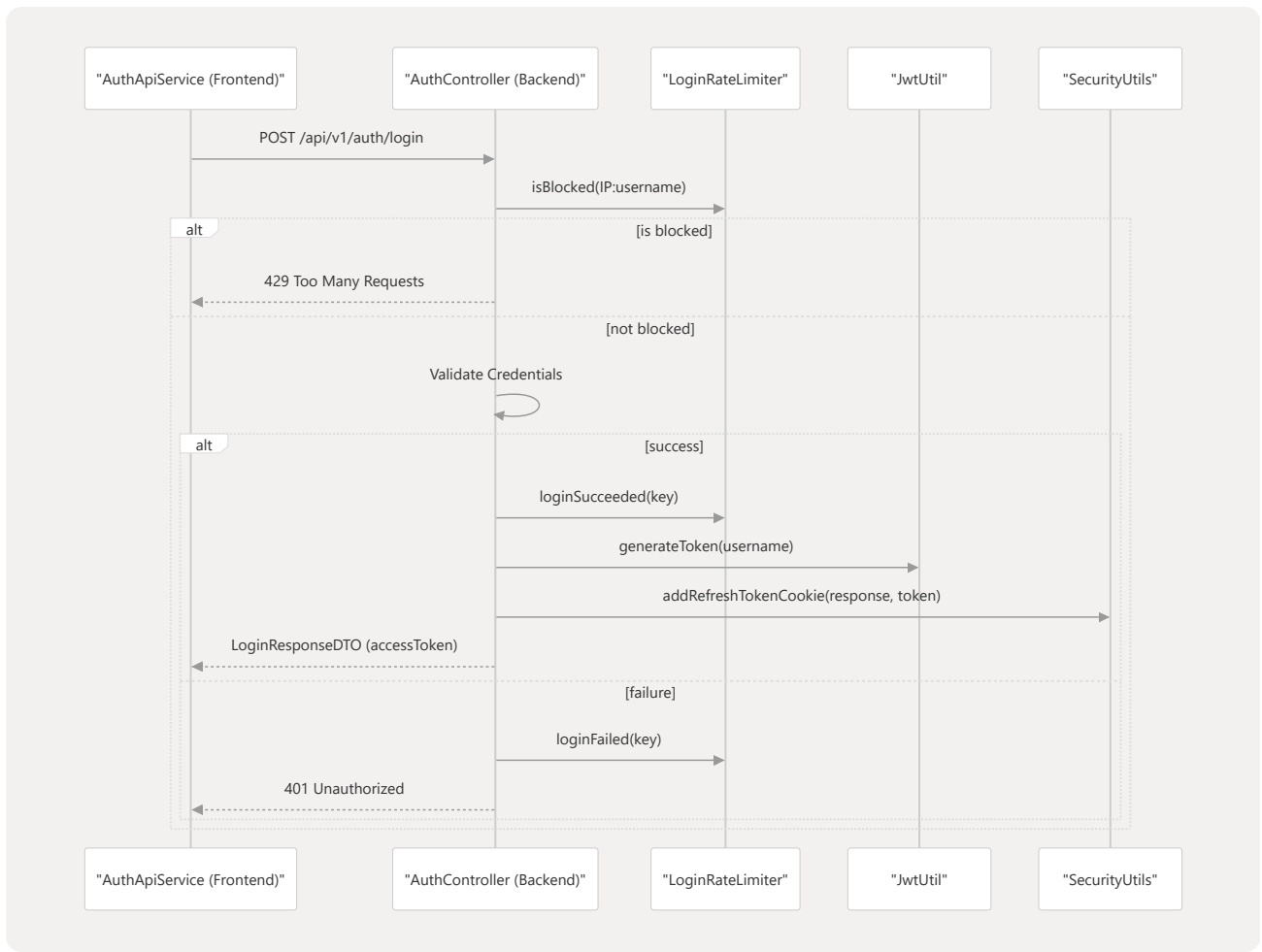
2. **Refresh Token:** Long-lived (default 7 days), stored in an `HttpOnly, SameSite=Lax` cookie named `refresh_token`

Backend/STUDER/src/main/java/facu/studer/security/SecurityUtils.java 20-64

Login Sequence

The following diagram illustrates the transition from user input to authenticated state.

Auth Flow: Code Entity Mapping



Sources: Frontend/STUDER/src/app/core/auth/auth-api.service.ts 25-27

Backend/STUDER/src/main/java/facu/studer/security/LoginRateLimiter.java 33-59

Backend/STUDER/src/main/java/facu/studer/security/SecurityUtils.java 49-64

Backend/STUDER/src/main/java/facu/studer/security/JwtUtil.java 44-63

Key Components

JwtUtil

Responsible for the lifecycle of JWTs. It uses a secret key defined by the `JWT_SECRET` environment variable Backend/STUDER/src/main/java/facu/studer/security/JwtUtil.java 33-35

- `generateToken(username, roles)` : Creates a signed token with `jti`, `sub`, and `roles` claims Backend/STUDER/src/main/java/facu/studer/security/JwtUtil.java 54-63
- `isTokenValid(token, username)` : Validates the token signature and expiration Backend/STUDER/src/main/java/facu/studer/security/JwtUtil.java 113-116

JwtAuthenticationFilter

A `OncePerRequestFilter` that intercepts every request to the backend.

1. Extracts the `Authorization` header

Backend/STUDER/src/main/java/facu/studer/security/JwtAuthenticationFilter.java 37

2. Parses the JWT and extracts the username and roles

Backend/STUDER/src/main/java/facu/studer/security/JwtAuthenticationFilter.java 46-55

3. Sets the `SecurityContextHolder` with a `UsernamePasswordAuthenticationToken` if valid

Backend/STUDER/src/main/java/facu/studer/security/JwtAuthenticationFilter.java 61-70

LoginRateLimiter

Provides protection against brute-force attacks using an in-memory `ConcurrentHashMap`

Backend/STUDER/src/main/java/facu/studer/security/LoginRateLimiter.java 25

- **Threshold:** 5 failed attempts

Backend/STUDER/src/main/java/facu/studer/security/LoginRateLimiter.java 17

- **Penalty:** 10-minute block

Backend/STUDER/src/main/java/facu/studer/security/LoginRateLimiter.java 21

- **Keying:** Attempts are tracked by a composite key of `IP:username`

Backend/STUDER/src/main/java/facu/studer/security/LoginRateLimiter.java 23

Sources: Backend/STUDER/src/main/java/facu/studer/security/JwtUtil.java 1-141

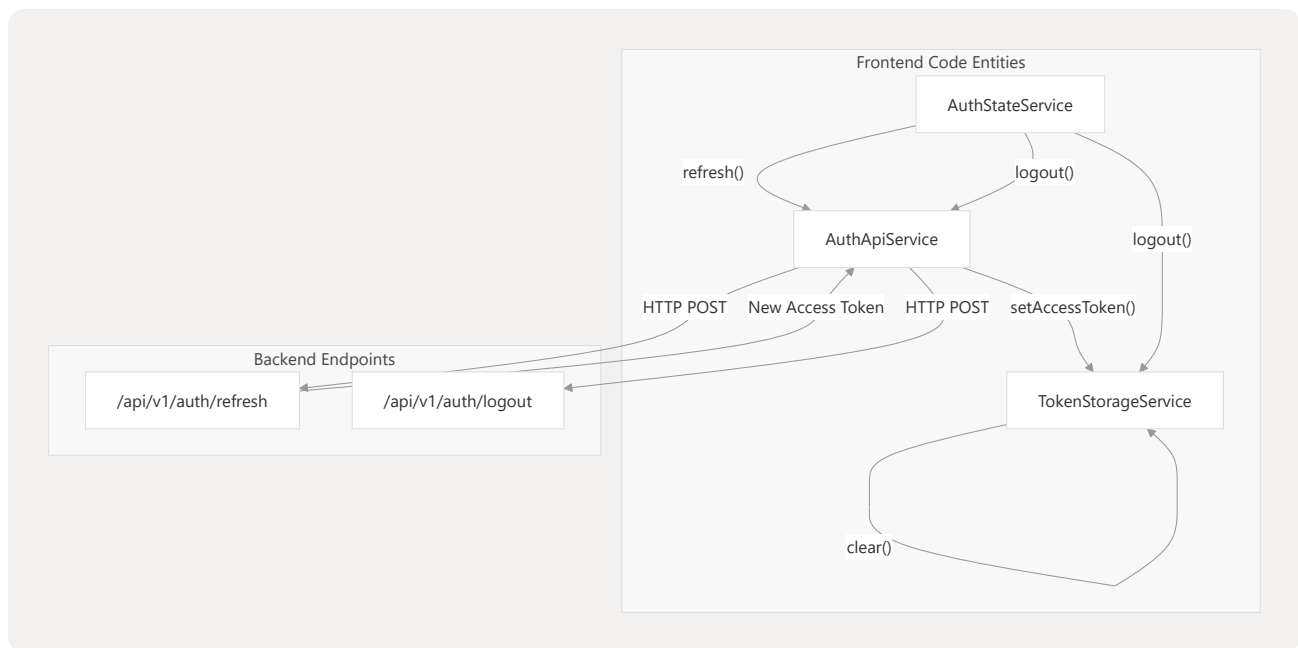
Backend/STUDER/src/main/java/facu/studer/security/JwtAuthenticationFilter.java 1-81

Backend/STUDER/src/main/java/facu/studer/security/LoginRateLimiter.java 1-74

Token Management (Frontend)

The frontend manages auth state via the `AuthStateService` and persists the short-lived access token using `TokenStorageService`.

Token Refresh and State Management



Auth Controllers & Endpoints

Endpoint	Method	Description	Security
<code>/api/v1/auth/login</code>	POST	Authenticates user, returns Access Token and sets Refresh Cookie.	Public
<code>/api/v1/auth/refresh</code>	POST	Uses Refresh Cookie to issue a new Access Token.	Public (Cookie-based)
<code>/api/v1/auth/logout</code>	POST	Clears the Refresh Cookie and local storage.	Authenticated
<code>/api/v1/users/register</code>	POST	Creates a new user account.	Public

Sources: Frontend/STUDER/src/app/core/auth/auth-state.service.ts 1-73

Frontend/STUDER/src/app/core/auth/auth-api.service.ts 1-39

Backend/STUDER/src/main/java/facu/studer/security/SecurityUtils.java 69-76